

WEBGL Cheat Sheet

Math Helpers (JS Math)

Common math functions.

```
Math.PI          // π
Math.cos(a)      // cos(a)
Math.sin(a)      // sin(a)
Math.tan(a)      // tan(a)
Math.hypot(x,y,z) //  $\sqrt{x^2 + y^2 + z^2}$ 
```

WebGL uses **radians**. Angle conversion (degrees → radians):

```
let rad = deg * Math.PI / 180;
```

Matrix Functions

These are basic tools for 3D transformations:

- `multiply(a, b)`
 - Multiplies two 4×4 matrices: `out = a * b`
 - Used to combine **Model**, **View**, **Projection** → MVP.
- `normalize(v)`
 - Turns any vector into a **unit vector** (length = 1).
 - Used when calculating camera direction vectors.
- `cross(a, b)`
 - Produces a vector perpendicular to both **a** and **b**.
 - Used to build camera axes for right, up, forward.
- `dot(a, b)`
 - Dot product
- `identity()`
 - Returns 4×4 identity matrix (no transform).
 - Base model matrix.

Canvas & WebGL2 Setup

```
const canvas = document.getElementById("glCanvas");
gl = canvas.getContext("webgl2");
program = initShaders(gl, "vertex-shader", "fragment-shader");
```

- **canvas** → HTML <canvas> where WebGL draws.
- **gl** → WebGL2 context (all rendering commands use this).
- **program** → Combined vertex + fragment shaders (GPU program).

WebGL will not draw anything until a shader program is created and used.

MVP & Uniform

You will send a single **MVP** matrix to the vertex shader:

```
mvpLoc = gl.getUniformLocation(program, "uMVP");
```

Build it as: $MVP = P \times V \times M$

Later in drawScene():

```
gl.uniformMatrix4fv(mvpLoc, false, mvp);
```

3D Object with VAO

To store vertex attribute setup (positions + colors):

```
vao = gl.createVertexArray();
gl.bindVertexArray(vao);
// bind + bufferData for positions, colors, and vertexAttribPointer(...)
gl.bindVertexArray(null);
```

And when drawing:

```
gl.bindVertexArray(vao);
gl.drawArrays(gl.TRIANGLES, 0, 18); // pyramid has 18 vertices
```

Camera & Perspective Parameters

You will control the camera using these parameters:

- fov – field of view (degrees, vertical) (zoom in/out)
- aspect – width / height of canvas
- near – near clipping plane (zNear)
- far – far clipping plane (zFar)
- radius – distance of camera from origin
- theta – horizontal angle (around Y axis) (left–right)
- phi – vertical angle (up/down)

Perspective matrix:

```
let proj = perspective(fov, aspect, near, far);
```

Eye position (spherical camera idea):

- theta → rotates camera **around** the object horizontally.
- phi → tilts camera up/down.
- radius → how far camera is from the origin.

Render Loop

To animate rotation and update camera every frame:

```
function render(time) {
    // update angle, camera, etc.
```

```
        drawScene();
        requestAnimationFrame(render);
    }
    // start:
    requestAnimationFrame(render);
```

- time is given by requestAnimationFrame.
- Use it to compute deltaTime and update rotation angle.

Keyboard Input (e.g. toggle pointer lock / modes)

Reading keys from keyboard. Used for features such as:

- toggling pointer lock
- resetting camera
- switching control modes

```
document.addEventListener(.....);
```

Mouse Move & Mouse Click

Mouse Move : Used to update theta and phi when pointer lock is active.

```
canvas.addEventListener(.....);
```

Mouse Click: Often used to enable pointer lock:

```
canvas.addEventListener(.....);
```